

API DOCUMENTATION

Health Care Plus

An Intelligent Machine Learning-Based Disease Prediction Web Application

Project Type	Major Project
Domain	Artificial Intelligence & Machine Learning
Backend Framework	Flask
Machine Learning Model	Random Forest Classifier

1. API Overview

Health Care Plus uses a Flask-based backend API to establish communication between the frontend web application and the trained Machine Learning model. The API receives user-selected symptoms, processes the input, and uses a Random Forest Classifier to generate a possible disease prediction.

The backend provides three main API endpoints:

Endpoint	Purpose
/	Checks the API status
/symptoms	Provides the list of available symptoms
/predict	Processes selected symptoms and generates a possible disease prediction

The API is designed to support seamless communication between the frontend interface and the Machine Learning-based prediction system.

2. Technology Stack

Component	Technology
Programming Language	Python
Backend Framework	Flask
Machine Learning Model	Random Forest Classifier
Data Processing	Pandas
Model Loading	Joblib
Data Format	JSON
Cross-Origin Support	Flask-CORS

3. API Endpoints

3.1 Home / API Status

Endpoint: /

HTTP Method: GET

Purpose

This endpoint checks whether the Symptom Checker API is running successfully. It returns the current API status and the model accuracy calculated by the backend, when available.

Example Response

```
{
  "message": "Symptom Checker API is running successfully!",
  "status": "success",
  "model_accuracy": 98.2
}
```

Note: The model_accuracy value is generated by the application and may vary depending on the trained model and testing data.

3.2 Get Available Symptoms

Endpoint: /symptoms

HTTP Method: GET

Purpose

This endpoint provides the list of symptoms available in the training dataset. It also returns the total number of available symptoms.

Example Response

```
{
  "symptoms": [
    "symptom_1",
    "symptom_2",
    "symptom_3"
  ],
  "count": 3
}
```

Note: The actual response contains the complete list of symptoms loaded from the training dataset.

4. Disease Prediction API

4.1 Prediction Endpoint

Endpoint: /predict

HTTP Method: POST

Purpose

The /predict endpoint is the primary Machine Learning API of the application. It receives selected symptoms from the frontend, processes them through the Flask backend, and sends the input to the trained Random Forest Classifier. The model analyzes the provided symptoms and generates a possible disease prediction along with additional prediction information.

4.2 Request Format

The API expects a JSON request containing a list of selected symptoms.

Example Request

```
{
  "symptoms": [
    "symptom_1",
    "symptom_2",
    "symptom_3"
  ]
}
```

The symptoms provided in the request are checked against the available symptoms loaded from the training dataset before being processed by the Machine Learning model.

4.3 Request Processing

1. The user selects symptoms through the frontend interface.
2. The frontend sends the selected symptoms to the /predict endpoint.
3. The Flask backend receives the JSON request.
4. The selected symptoms are converted into a Machine Learning feature vector.
5. The Random Forest Classifier processes the input.
6. The model generates a possible disease prediction.
7. Prediction probabilities are calculated.
8. The top three possible predictions are identified.
9. Relevant disease information is retrieved.
10. The final response is returned to the frontend.

4.4 Response Format

A successful prediction response contains:

- Prediction status
- Predicted disease
- Prediction confidence
- Model accuracy
- Top three predictions
- Selected symptoms
- Disease information
- Medical disclaimer

Example Response

```
{
  "status": "success",
  "predicted_disease": "Disease Name",
  "confidence": 95.5,
  "model_accuracy": 98.2,
  "top_predictions": [
    {
      "disease": "Disease Name",
      "probability": 95.5
    },
    {
      "disease": "Disease Name 2",
      "probability": 3.2
    },
    {

```

```

    "disease": "Disease Name 3",
    "probability": 1.3
  }
],
"selected_symptoms": [
  "symptom_1",
  "symptom_2"
],
"disease_information": {
  "description": "Information about the predicted condition.",
  "precautions": "General precautionary information.",
  "recommended_specialist": "General Physician"
},
"disclaimer": "This tool is for educational and informational purposes only. It is not a substitute for professional medical diagnosis, treatment, or advice."
}

```

5. Error Handling

The API includes validation and error handling mechanisms to manage invalid, incomplete, or unexpected requests.

5.1 No Data Provided

HTTP Status Code: 400 Bad Request

Returned when the request does not contain any data.

```
{"status": "error", "message": "No data provided."}
```

5.2 No Symptom Selected

HTTP Status Code: 400 Bad Request

Returned when no symptoms are provided in the request.

```
{"status": "error", "message": "Please select at least one symptom."}
```

5.3 No Valid Symptoms

HTTP Status Code: 400 Bad Request

Returned when the provided symptoms do not match the available symptoms in the dataset.

```
{"status": "error", "message": "No valid symptoms were selected."}
```

5.4 Internal Server Error

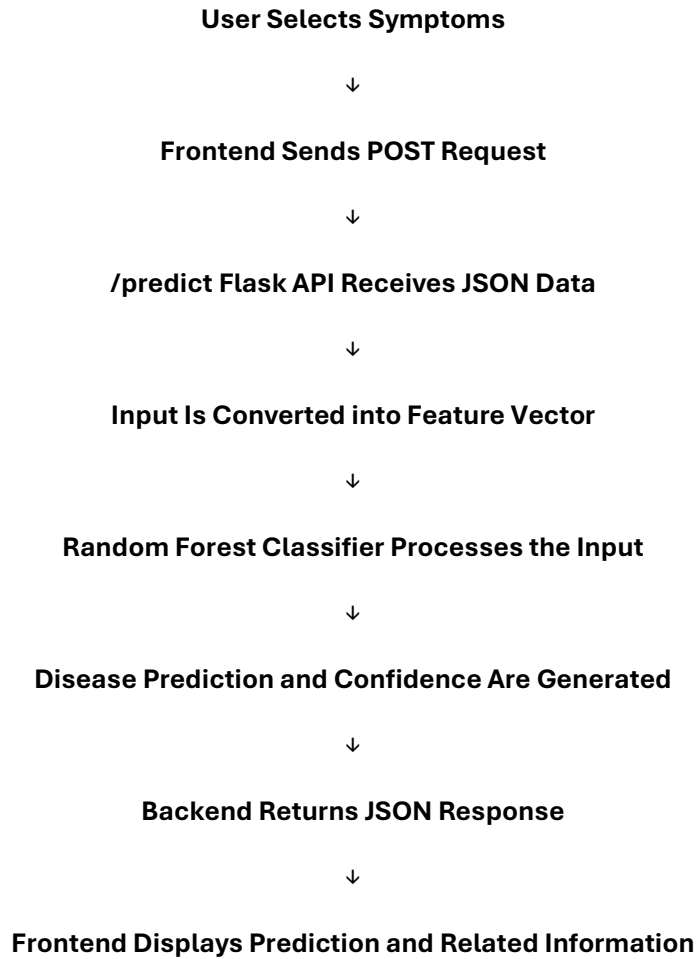
HTTP Status Code: 500 Internal Server Error

Returned when an unexpected error occurs while processing the prediction request.

```
{"status": "error", "message": "Error message"}
```

6. API Workflow

The complete API workflow is:



7. API Summary

Endpoint	HTTP Method	Purpose
/	GET	Checks API status and returns model accuracy
/symptoms	GET	Returns the list of available symptoms
/predict	POST	Processes selected symptoms and predicts a possible disease

8. Disclaimer

This application is developed for educational and informational purposes. The predictions generated by the system should not be considered a substitute for professional medical diagnosis, treatment, or advice.

Users should consult a qualified healthcare professional for proper medical evaluation and treatment.

— Document End —